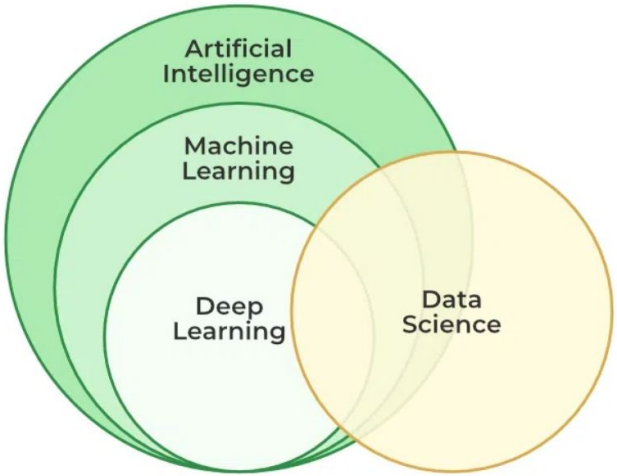
Classification

Lecturer: Albert Yang (楊景明)

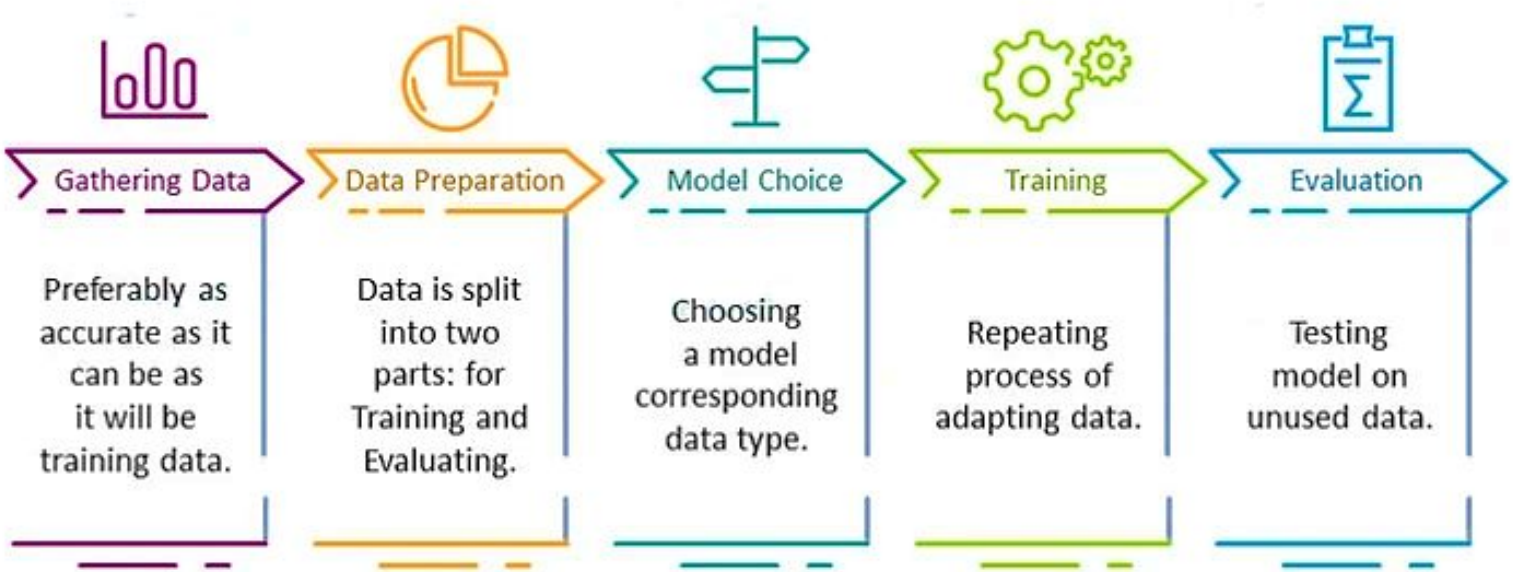
A Recap of Machine Learning

- Machine learning(ML) focused on developing algorithms and statistical models
- Allow computers to learn from data and make predictions or decisions
- ML systems gradually improve their accuracy as they are exposed to more data



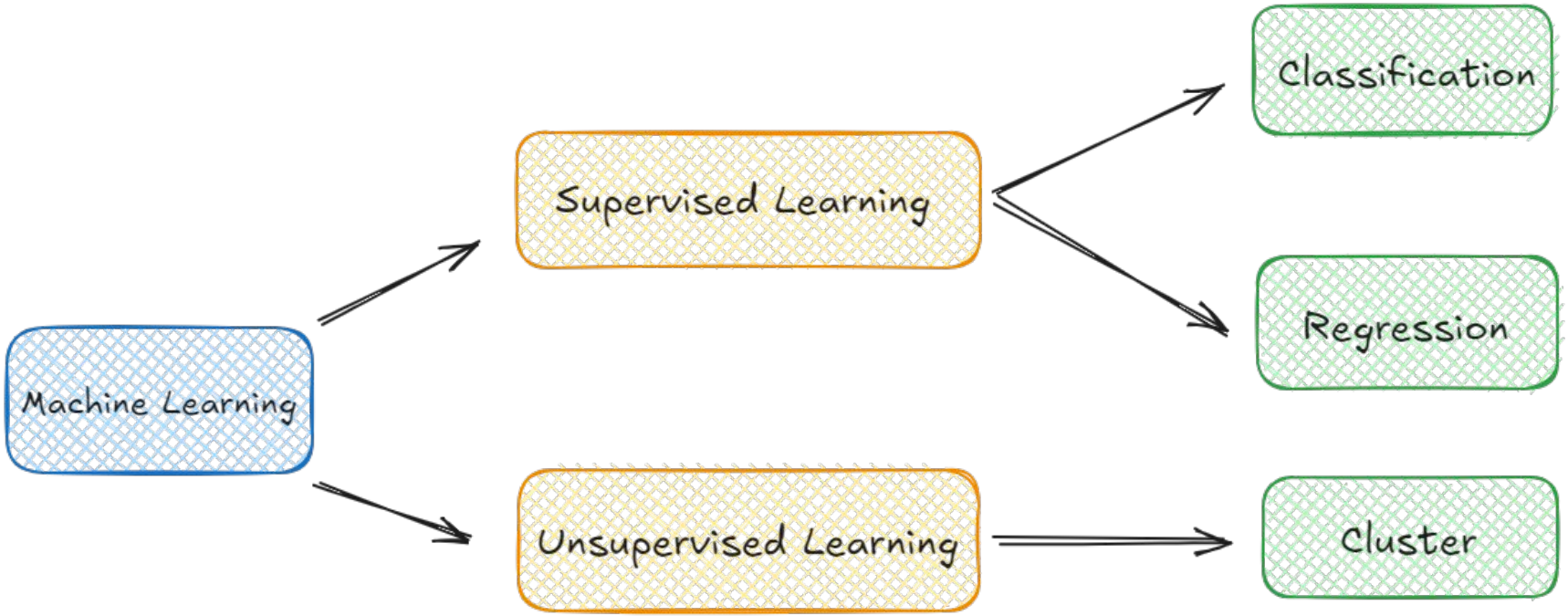
Source: https://reurl.cc/Dllma6

Machine Learning Process Outline Diagram



Source: https://www.infodiagram.com/s/knowledge%20learning/

Machine Learning Methods

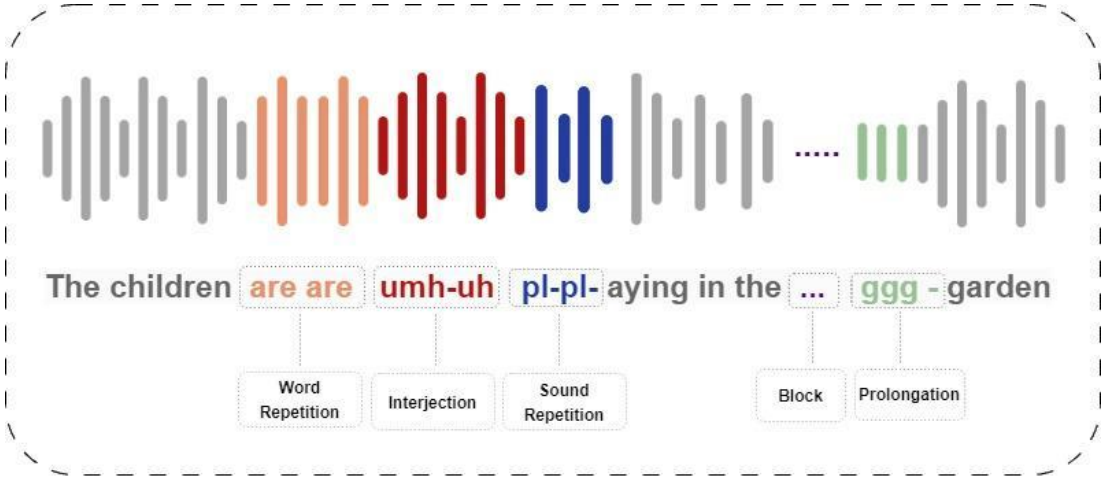


Implementation

- Download notebook from iLearning

What Are Classification Tasks?

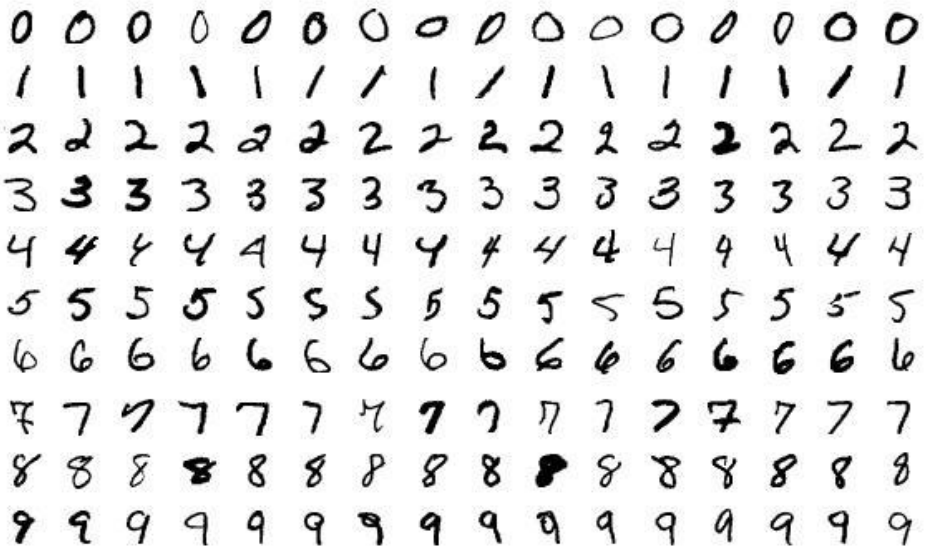
- A classification task involves categorizing data into distinct classes or groups based on specific features or patterns
  - Image Classification
  - Text Classification
  - Speech/Audio Classification



Source: https://reurl.cc/6dd5Vb

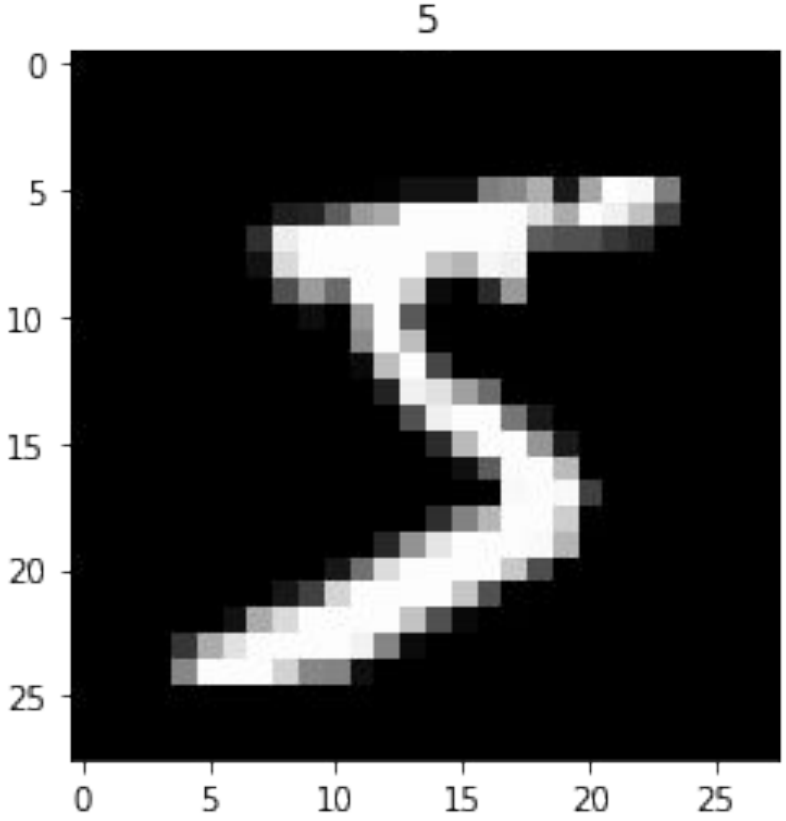
Example: MNIST Dataset

- Contains 70,000 images of hand-written digits
- A “Hello world” task in machine learning



Source: https://paperswithcode.com/dataset/mnist

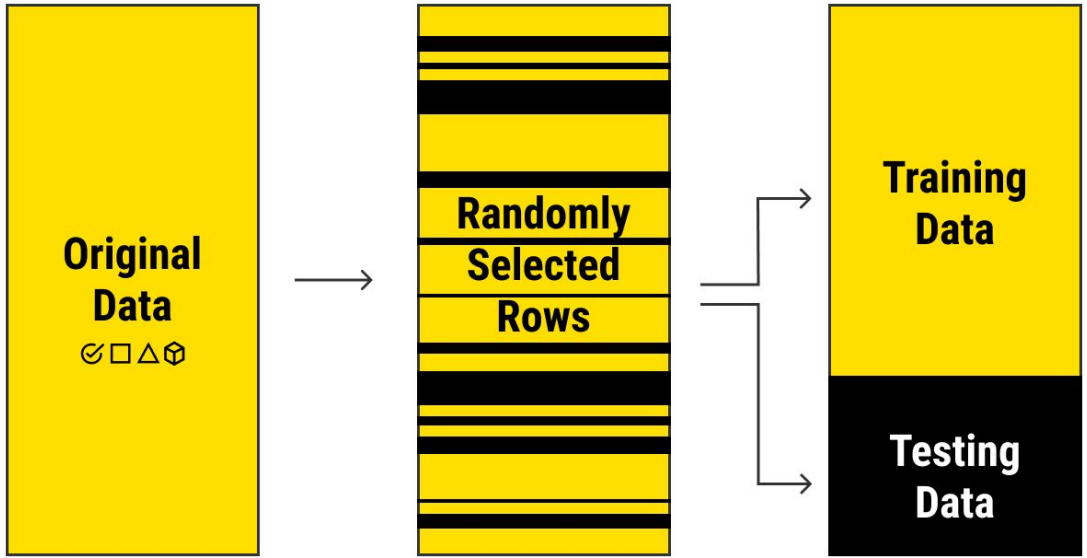
- 28 x 28 pixels
- 784 features or pixels (28 x 28)
- Each feature rerpresent the shape of color, from 0 to 255



Source: https://labelyourdata.com/articles/machine-learning-and-training-data

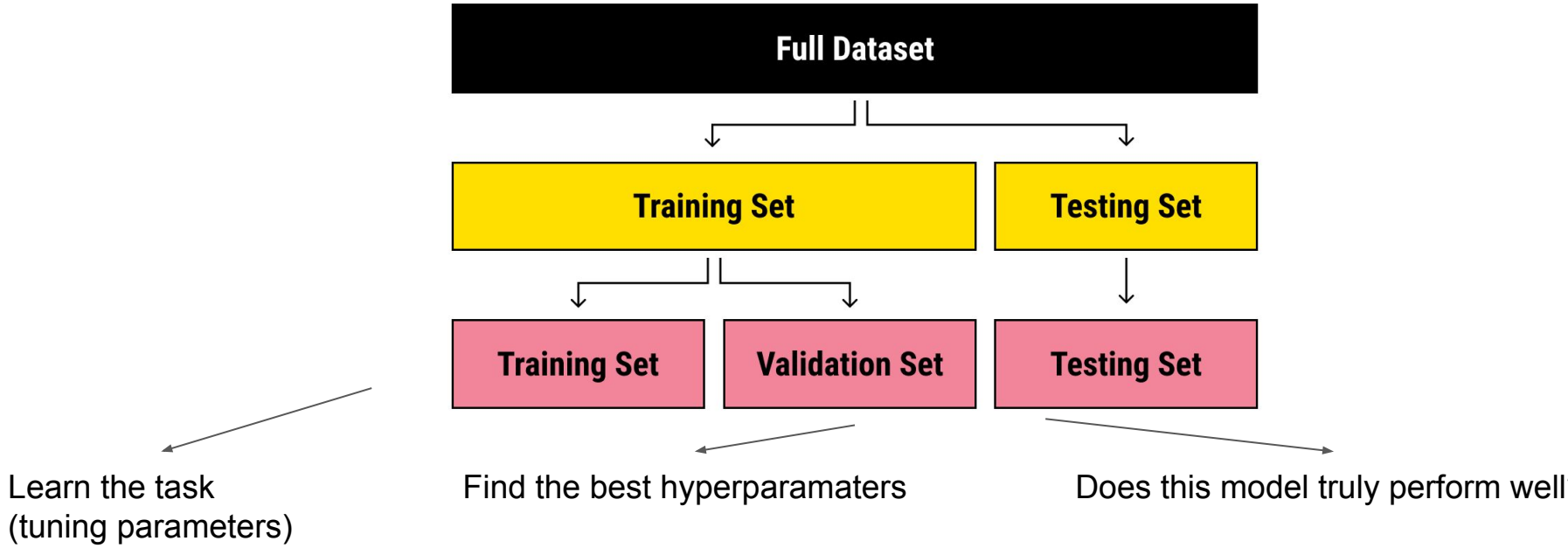
Datasets Split

- We need to split the data into two (or three) sets
  - Training and testing sets
  - Validation set



Source: https://labelyourdata.com/articles/machine-learning-and-training-data

- We need to split the data into two (or three) sets
  - Training and testing sets
  - Validation set



Why Do We Need Validation Set?

	Training set	Testing set
Model 1 (with A hyperparameter)	0.9	0.83
Model 2 (with B hyperparameter)	0.9	
Model 3 (with C hyperparameter)	0.9	

Why Do We Need Validation Set?

	Training set	Testing set
Model 1 (with A hyperparameter)	0.9	0.83
Model 2 (with B hyperparameter)	0.9	0.85
Model 3 (with C hyperparameter)	0.9	

Why Do We Need Validation Set?

	Training set	Testing set
Model 1 (with A hyperparameter)	0.9	0.83
Model 2 (with B hyperparameter)	0.9	0.85
Model 3 (with C hyperparameter)	0.9	**0.87**

Why Do We Need Validation Set?

	Training set	Testing set
Model 1 (with A hyperparameter)	0.9	0.83
Model 2 (with B hyperparameter)	0.9	0.85
Model 3 (with C hyperparameter)	0.9	**0.87**

- Does model 3 truly perfom well?

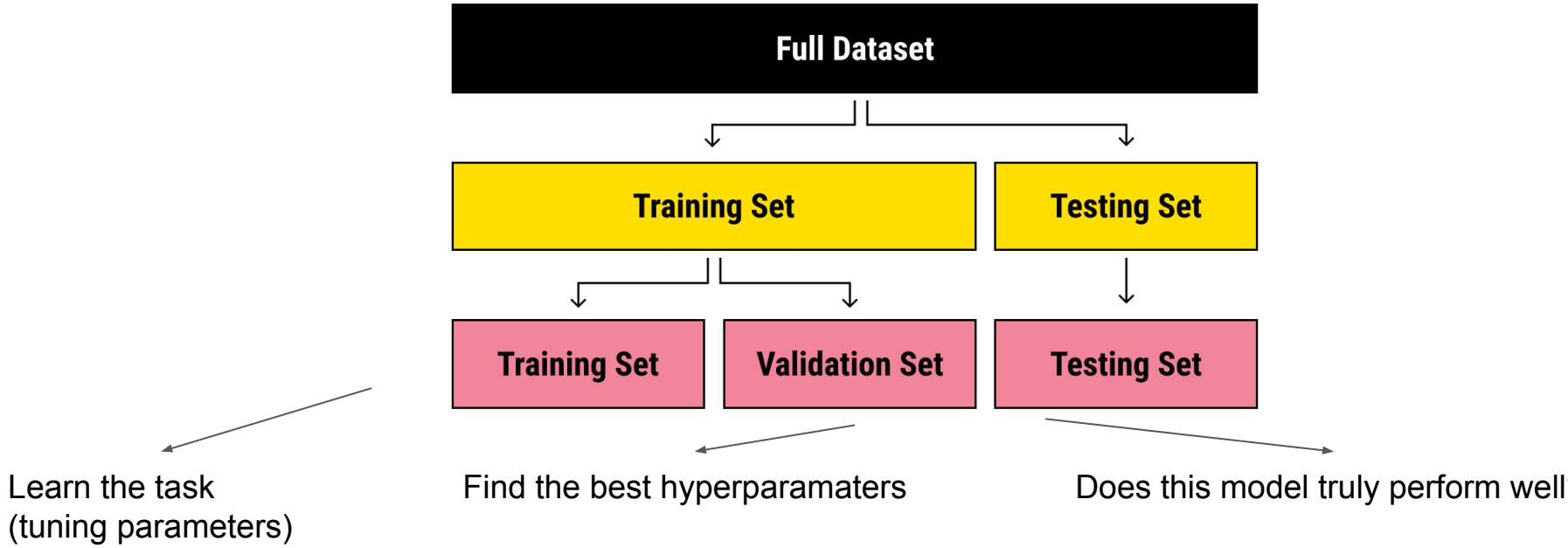
Why Do We Need Validation Set?

	Training set	Testing set
Model 1 (with A hyperparameter)	0.9	0.83
Model 2 (with B hyperparameter)	0.9	0.85
Model 3 (with C hyperparameter)	0.9	**0.87**

- Does model 3 truly perfom well?
- Not necessarily, because it might overfit on the testing set

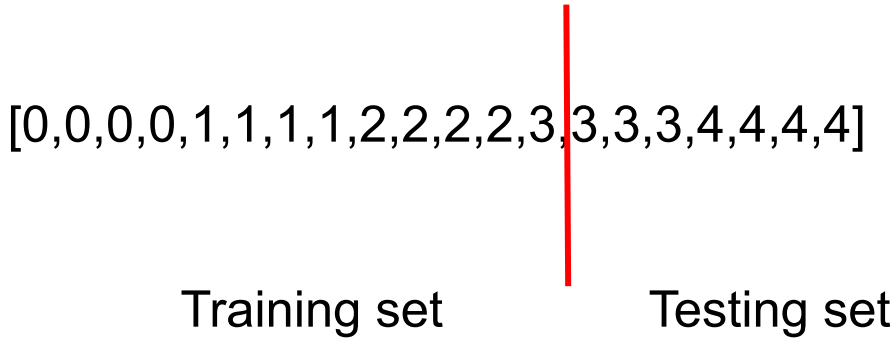
Source: https://labelyourdata.com/articles/machine-learning-and-training-data

- We need to split the data into two (or three) sets
  - Training and testing sets
  - Validation set



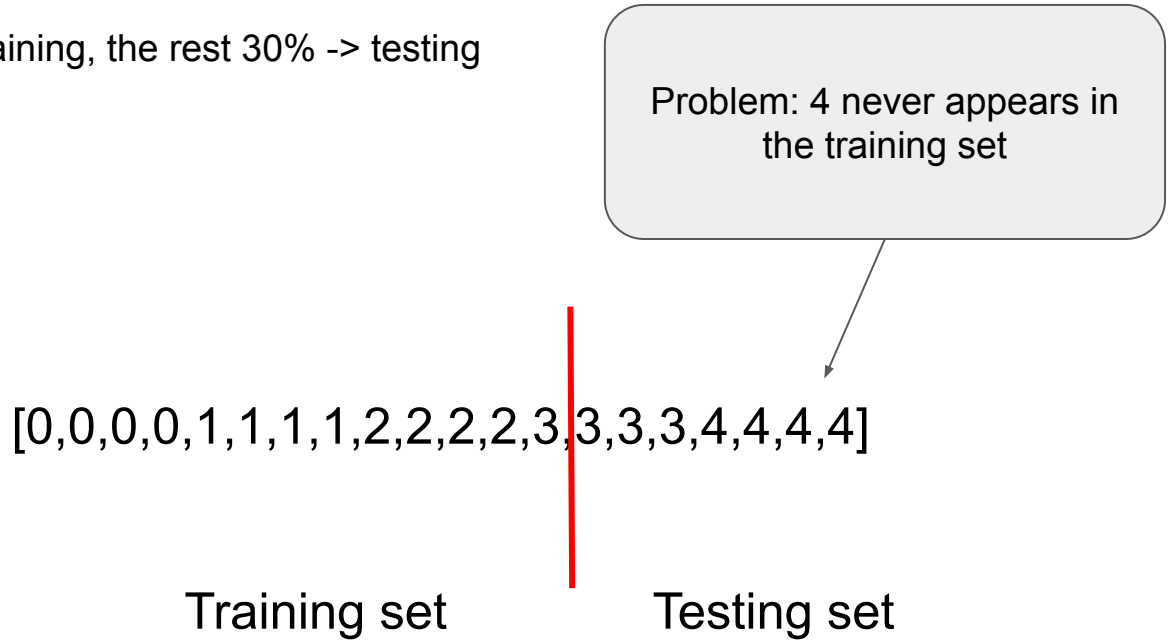
Training Set or Testing Set?

- One-cut
  - Ex: the first 70% data -> training, the rest 30% -> testing
- Random sampling
- Strafitied sampling



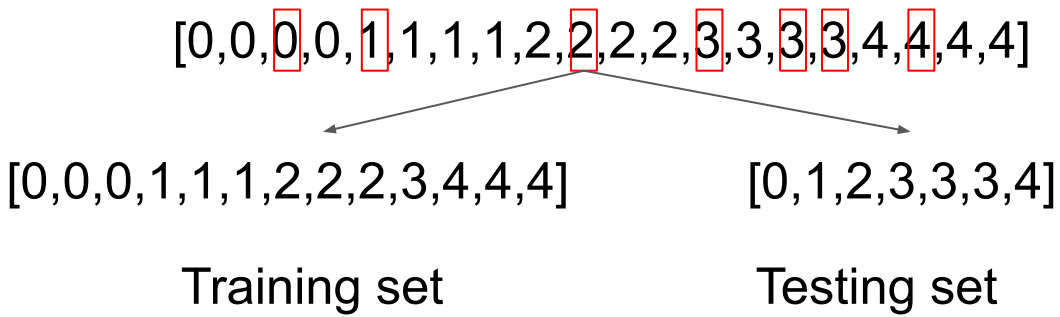
Training Set or Testing Set?

- One-cut
  - Ex: the first 70% data -> training, the rest 30% -> testing
- Random sampling
- Strafitied sampling



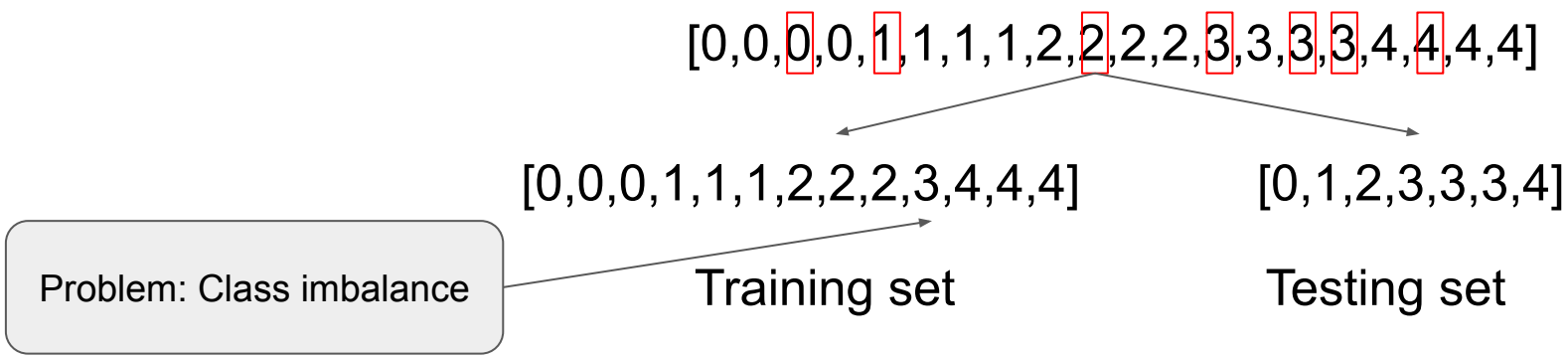
Training Set or Testing Set?

- One-cut
  - Ex: the first 70% data -> training, the rest 30% -> testing
- Random sampling
- Strafitied sampling



Training Set or Testing Set?

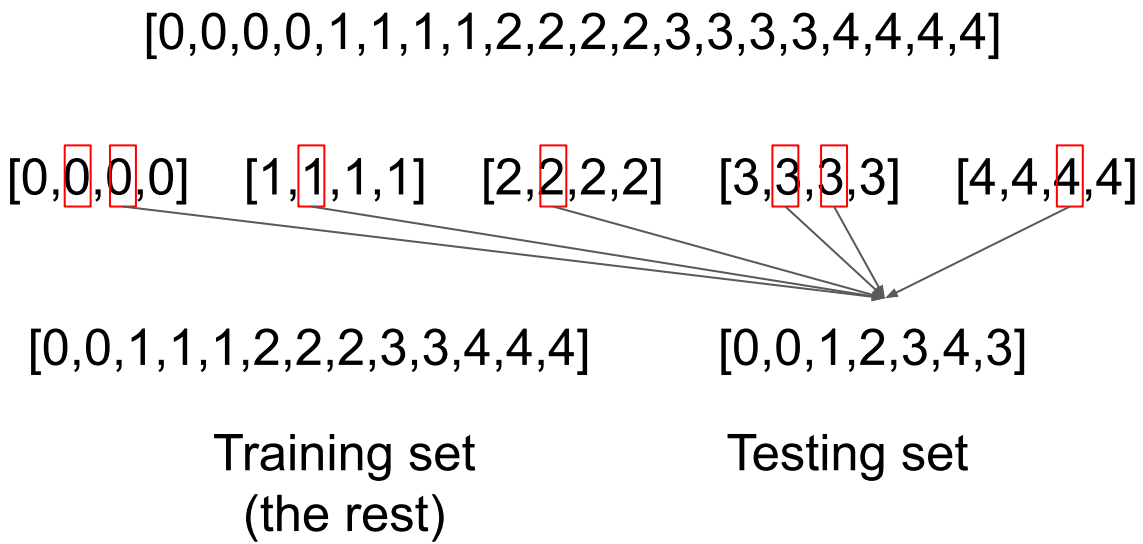
- One-cut
  - Ex: the first 70% data -> training, the rest 30% -> testing
- Random sampling
- Strafitied sampling



Training Set or Testing Set?

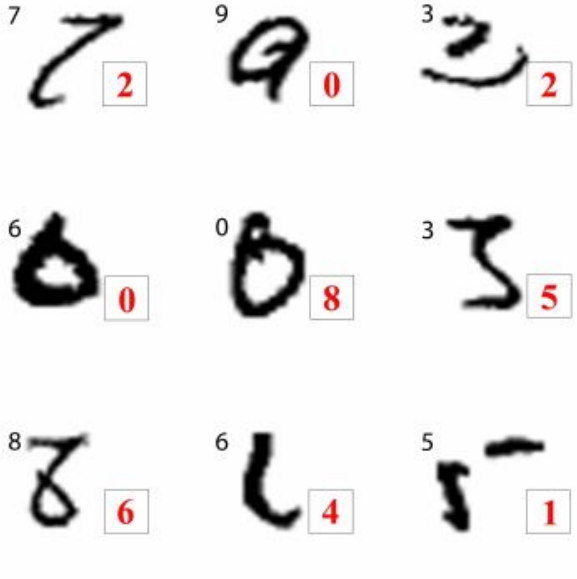
- One-cut
  - Ex: the first 70% data -> training, the rest 30% -> testing
- Random sampling
- Strafitied sampling

- Calculate the proportion of each class (ex: 25% for each)
- Decide size of testing set (ex: 6)
- Select the proportional sample (1.5) form each class



Unrepresentative Samples

- The features of some samples may not be representative of their categories
- The model may only learn from good features



[0,0,0,0,1,1,1,1,2,2,2,2,3,3,3,3,4,4,4,4]

[0,0,0,0] [1,1,1,1] [2,2,2,2] [3,3,3,3] [4,4,4,4]

[0,0,1,1,1,2,2,2,3,3,4,4,4]

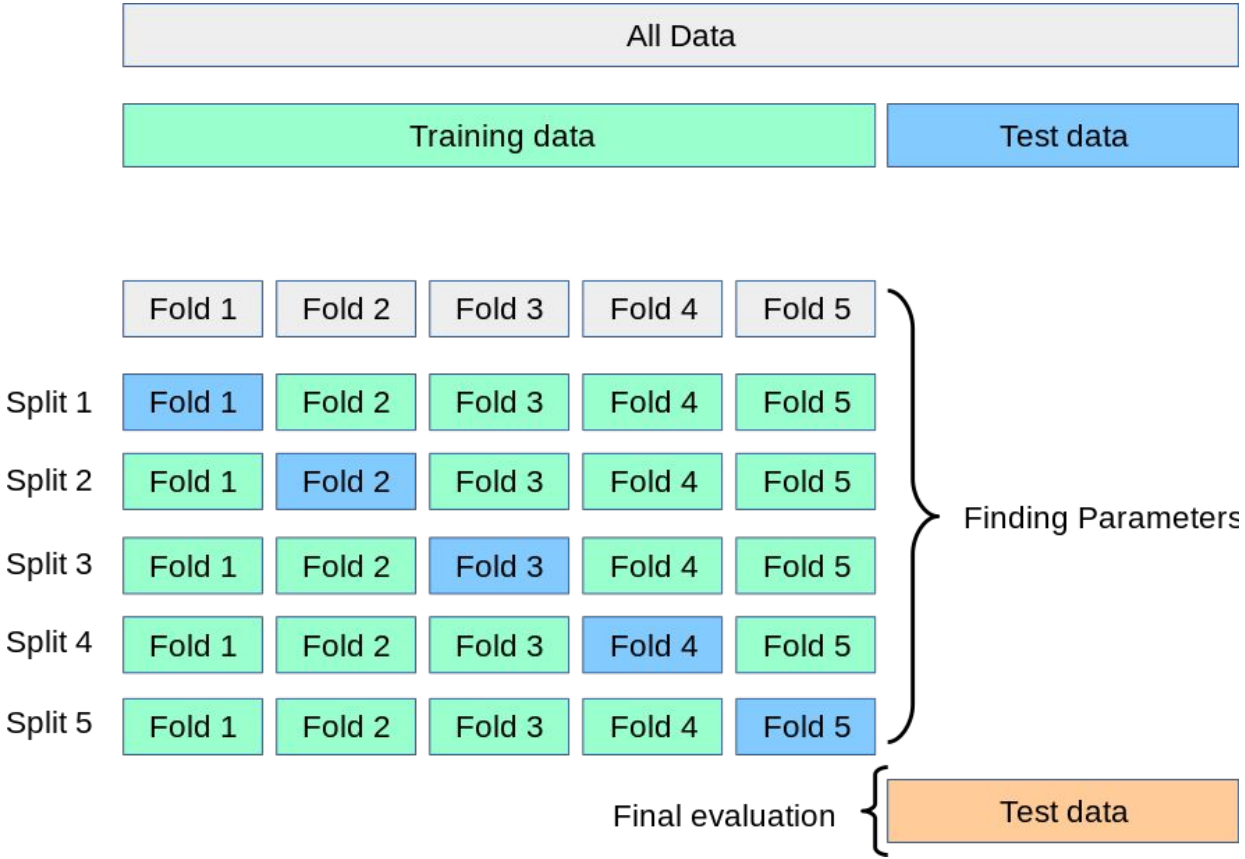
[0,0,1,2,3,4,3]

Training set
(the rest)

Testing set

Source: https://scikit-learn.org/stable/modules/cross_validation.html

K-fold Cross Validation



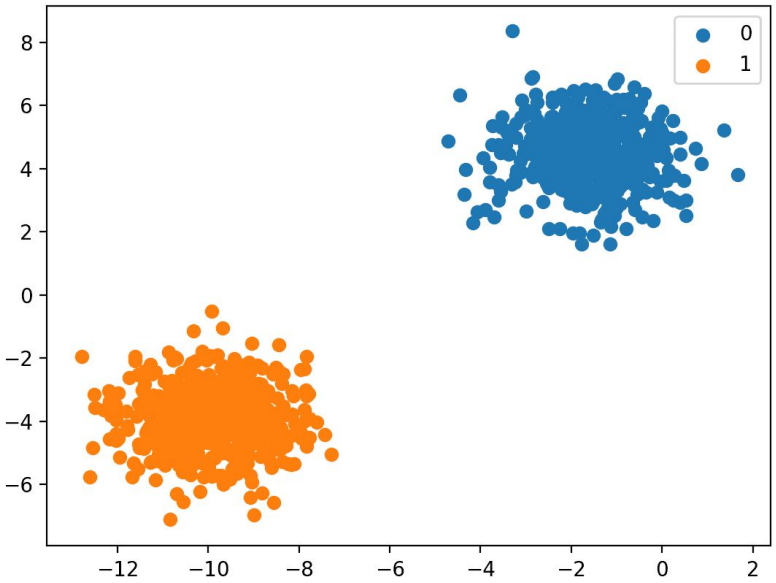
Classification

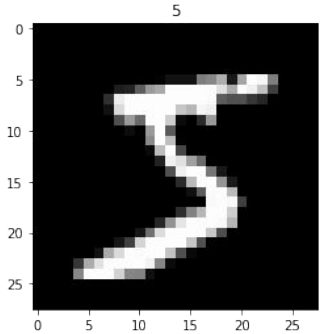
- Binary class classification
- Multiclass classification
- Multilabel classification
- Multioutput classification

Source: https://machinelearningmastery.com/types-of-classification-in-machine-learning/

Binary Classification

- Logistic regression
- K-nearest neighbors
- Decision trees
- Support vector machine
- Naive bayes





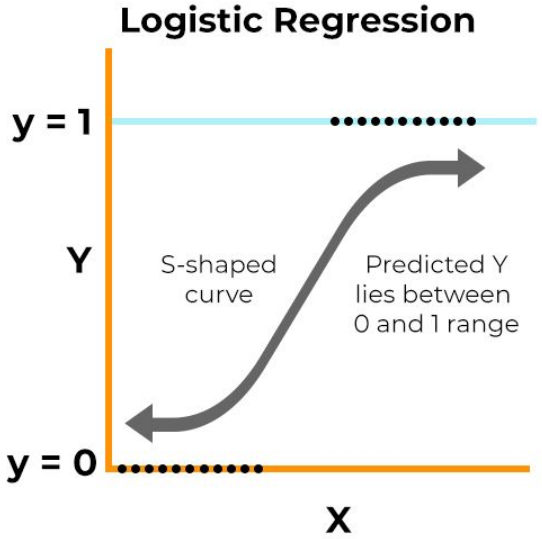
Model

is it 5?

Y/N

Logistic Regression

- Logistic regression is a statistical model
- Commonly used for classification and predictive analytics
- Particularly in binary classification tasks
- The output is restricted to values between 0 and 1



Source: https://reurl.cc/nvvDyd

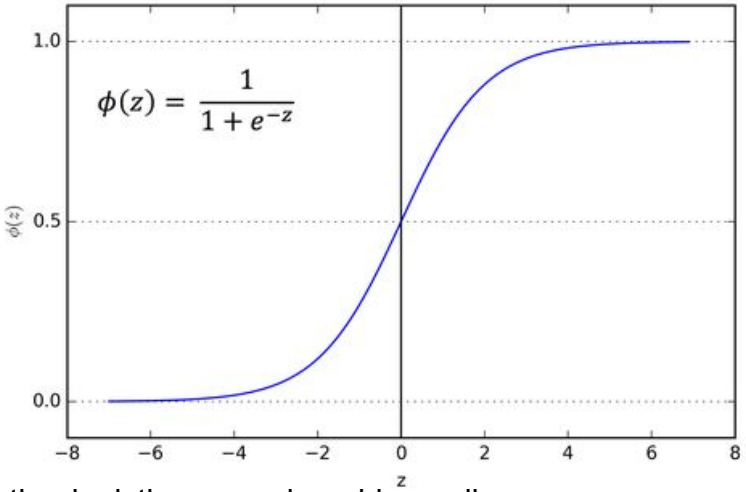
Logistic Regression

- Binary classification:
  - Predicts probabilities and assigns data to class 0 or 1 based on a threshold.
  - The output is a probability between 0 and 1
  - If the probability exceeds the threshold (e.g., 0.5), it's classified as 1; otherwise, as 0
  - The threshold can be adjusted
  - It finds a linear decision boundary that separates the two classes
- Multiclass classification:
  - Logistic regression can be extended to handle multi-class classification problems using softmax function (softmax function will be taught in later chapters)

Sigmoid Function

- Logistic regression uses a sigmoid function to map any real-valued number into a value between 0 and 1
- It approaches 0 as the input becomes negative and 1 as the input becomes positive. When the input is 0, the sigmoid function returns 0.5

$$S(x) = \frac{1}{1 + e^{-x}}$$



Source: https://www.linkedin.com/pulse/understanding-sigmoid-function-logistic-regression-piduguralla

Linear Regression vs Logistic Regression

	Linear Regression	**Logistic Regression**
Purpose	Used for regression tasks (predicting continuous values)	Used for classification tasks (e.g., binary classification)
Output	Predicts continuous values (e.g., 0.6)	Predicts probabilities (between 0 and 1)
Target Variable	Continuous (e.g., house prices, temperature)	Categorical (e.g., 0/1, spam/not spam)

Advantages of Logistic Regression

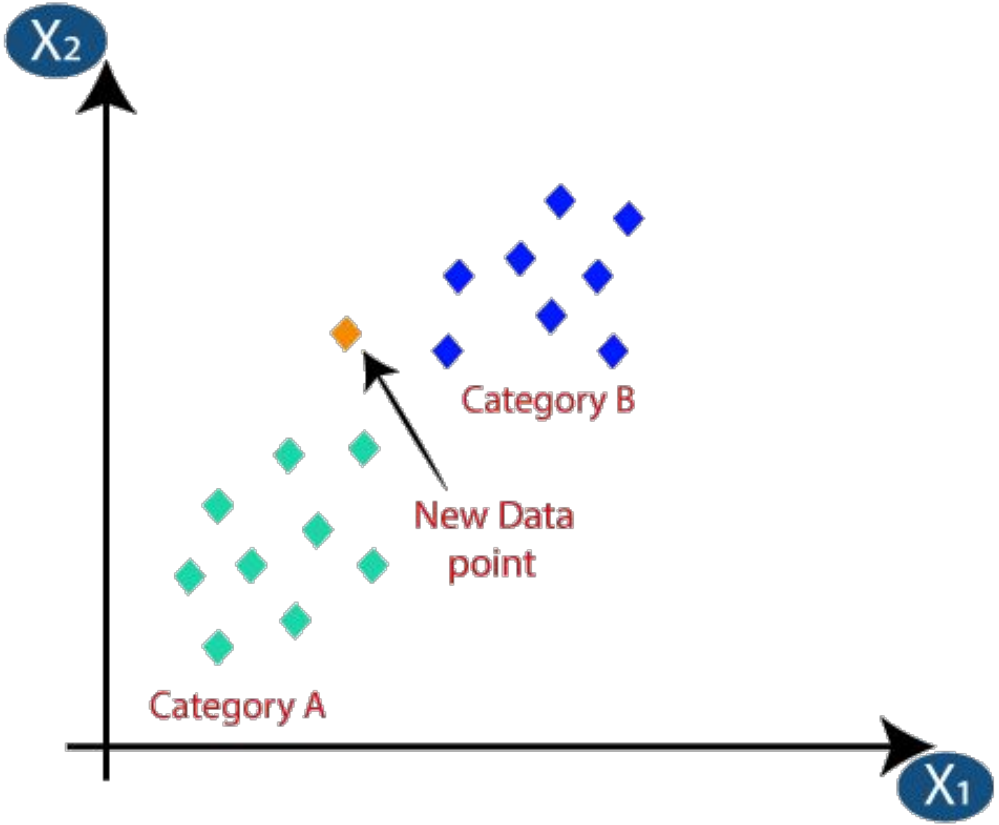
- Easy to use, fast classification
- No strong distributional assumptions, but assumes linearity
- Handles multiple classes:
  - Extensions like multinomial logistic regression
- Predictor insights:
  - Shows how important a feature is and its effect (positive or negative)
- Performs well when the classes are linearly separable

Disadvantages of Logistic Regression

- Linear boundaries:
  - Only works with linear data, so it's not good for complex patterns
- Cannot handle feature interactions
  - Assume features are independent of each other
- Prone to underfitting
- Sensitive to outliers

Source: https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning

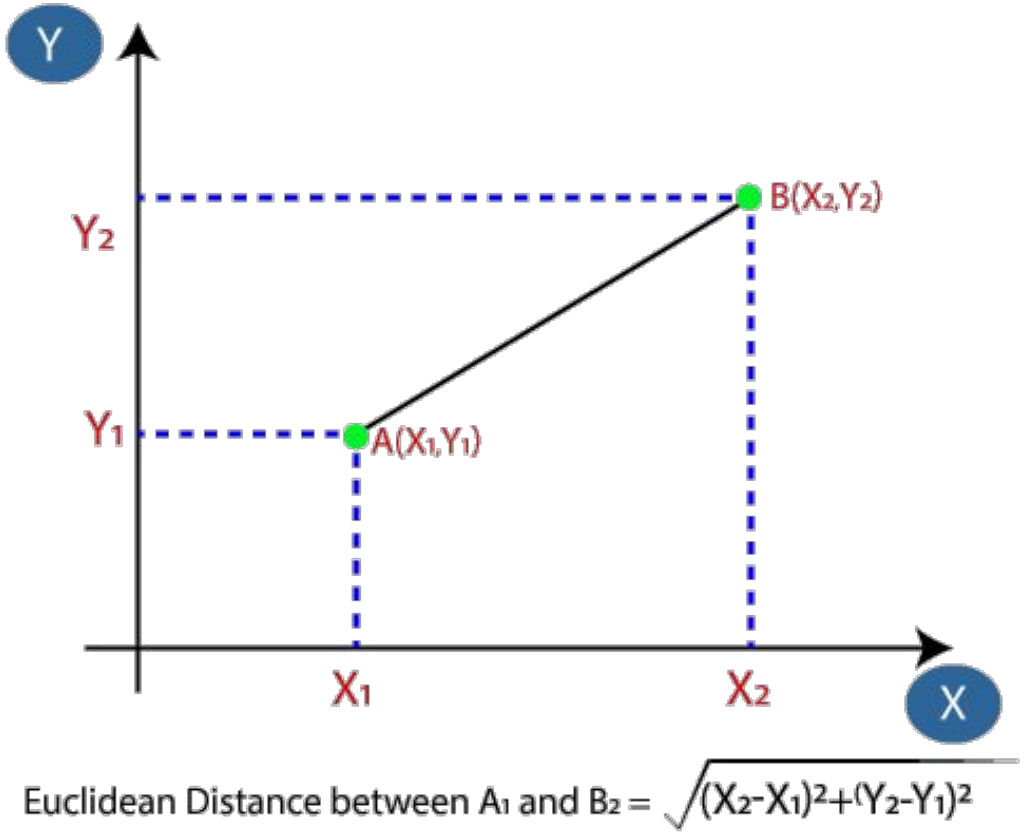
K-Nearest Neighbors



- Step 1: Select the number K of the neighbors, let's choose 5

Source: https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning

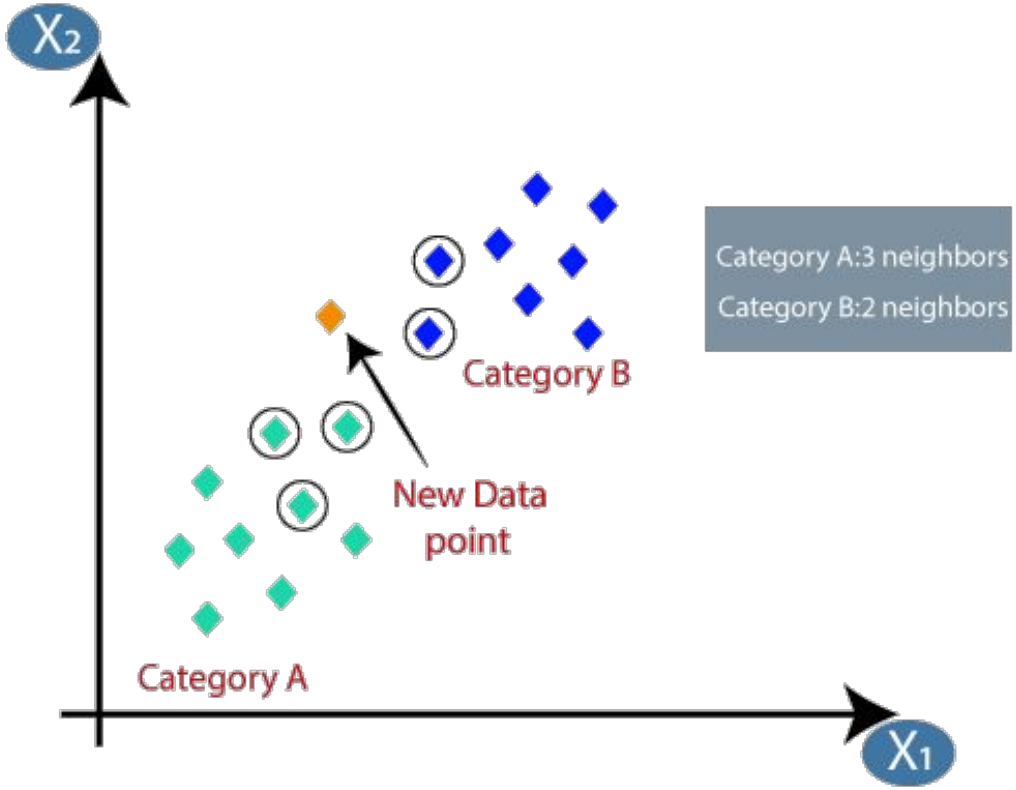
K-Nearest Neighbors



- Step 1: Select the number K of the neighbors, let's choose 5
- Step 2: Calculate the Euclidean distance between the data points

Source: https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning

K-Nearest Neighbors



- Step 1: Select the number K of the neighbors, let's choose 5
- Step 2: Calculate the Euclidean distance between the data points
- Step 3: Take the K nearest neighbors
- Step 4: Among these k neighbors, count the number of the data points in each category.

Advantages of KNN

- Easy to understand
- Versatile for a variety of datasets
- Handle multi-class classification problems
- Adaptable to different distance metrics:
    - e.g., Euclidean, Manhattan, Minkowski, allowing for customization based on the data

Disadvantages of KNN

- Computationally expensive
  - KNN is slow for large datasets as it computes the distance to all points during prediction
- Sensitive to noise and outliers
  - Irrelevant features or noisy points can have a significant impact on the classifications
- Can only predict distinct categories:
  - KNN does not provide probabilistic outputs, only hard classifications.
- Determination of K:
  - The performance of KNN heavily depends on the choice of K

Naive Bayes





The probability of x being class C is P(C | x)

$$= \frac{P(x \mid C) * P(C)}{P(x)}$$

Naive Bayes





The probability of x being class C is P(C | x)

$$= \frac{P(x \mid C) * P(C)}{P(x)}$$

The probability of a new email with content c being a normal email N is P(N | c)

$$= \frac{P(c \mid N) * P(N)}{P(c)}$$

Naive Bayes





The probability of x being class C is P(C | x)

$$= \frac{P(x \mid C) * P(C)}{P(x)}$$

The probability of a new email with content c being a normal email N is P(N | c)

$$= \frac{P(c \mid N) * P(N)}{P(c)}$$

The probability of a new email with content c being a spam email S is P(S | c)

$$= \frac{P(c \mid S) * P(S)}{P(c)}$$

Naive Bayes



Dear: 8
Friend: 5
Lunch: 3
Money: 1

P(Dear | N) = 8/17 = 0.47
P(Friend | N) = 5/17 = 0.29
P(Lunch | N): 3/17 = 0.18
P(Money | N): 1/17 = 0.06



Dear: 2
Friend: 1
Lunch: 0
Money: 4

P(Dear | S) = 2/7 = 0.29
P(Friend | S) = 1/7 = 0.14
P(Lunch | S): 0/7 = 0
P(Money | S): 4/7 = 0.57

Naive Bayes



P(Dear | N) = 8/17 = 0.47
P(Friend | N) = 5/17 = 0.29
P(Lunch | N): 3/17 = 0.18
P(Money | N): 1/17 = 0.06

P(N) = 6/10 = 0.6



P(Dear | S) = 2/7 = 0.29
P(Friend | S) = 1/7 = 0.14
P(Lunch | S): 0/7 = 0
P(Money | S): 4/7 = 0.57

P(S) = 4/10 = 0.4

Naive Bayes

Dear Friend



P(Dear | N) = 8/17 = 0.47
P(Friend | N) = 5/17 = 0.29
P(Lunch | N): 3/17 = 0.18
P(Money | N): 1/17 = 0.06

P(N) = 6/10 = 0.6



P(Dear | S) = 2/7 = 0.29
P(Friend | S) = 1/7 = 0.14
P(Lunch | S): 0/7 = 0
P(Money | S): 4/7 = 0.57

P(S) = 4/10 = 0.4

P(N) * P(Dear | N) * P(Friend | N)
= 0.6 * 0.47 * 0.29
= 0.08 $\propto$ P(N | Dear Friend)

Naive Bayes

Dear Friend



P(Dear | N) = 8/17 = 0.47
P(Friend | N) = 5/17 = 0.29
P(Lunch | N): 3/17 = 0.18
P(Money | N): 1/17 = 0.06

P(N) = 6/10 = 0.6

P(N) * P(Dear | N) * P(Friend | N)
= 0.6 * 0.47 * 0.29
= 0.08 ∝ P(N | Dear Friend)



P(Dear | S) = 2/7 = 0.29
P(Friend | S) = 1/7 = 0.14
P(Lunch | S): 0/7 = 0
P(Money | S): 4/7 = 0.57

P(S) = 4/10 = 0.4

P(S) * P(Dear | S) * P(Friend | S)
= 0.4 * 0.29 * 0.14
= 0.01 ∝ P(S | Dear Friend)

Naive Bayes

Dear Friend



P(Dear | N) = 8/17 = 0.47
P(Friend | N) = 5/17 = 0.29
P(Lunch | N): 3/17 = 0.18
P(Money | N): 1/17 = 0.06

P(N) = 6/10 = 0.6

P(N) * P(Dear | N) * P(Friend | N)
= 0.6 * 0.47 * 0.29
= 0.08 ∝ P(N | Dear Friend)



P(Dear | S) = 2/7 = 0.29
P(Friend | S) = 1/7 = 0.14
P(Lunch | S): 0/7 = 0
P(Money | S): 4/7 = 0.57

P(S) = 4/10 = 0.4

P(S) * P(Dear | S) * P(Friend | S)
= 0.4 * 0.29 * 0.14
= 0.01 ∝ P(S | Dear Friend)

0.08 > 0.01 –> Normal

Advantages of Naive Bayes

- Simple to implement, very fast
- Scales well:
  - If the conditional independence assumption holds, it could give great results
  - Usually not the case :(
- Can handle high-dimensional data
  - Treats each feature as independent

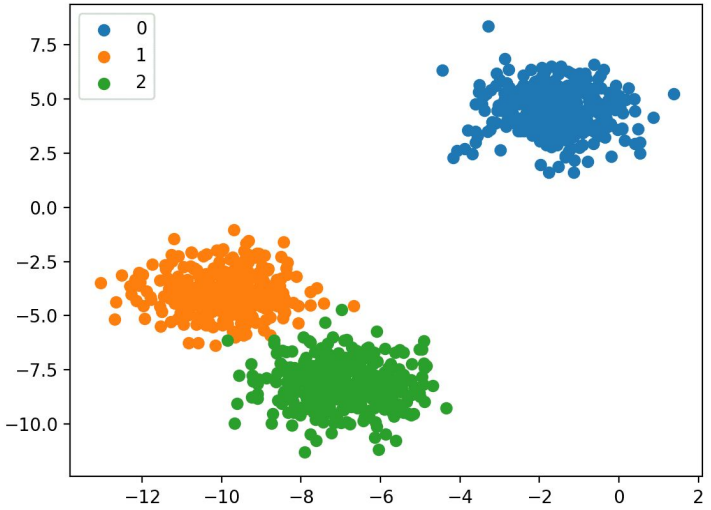
Disadvantages of Naive Bayes

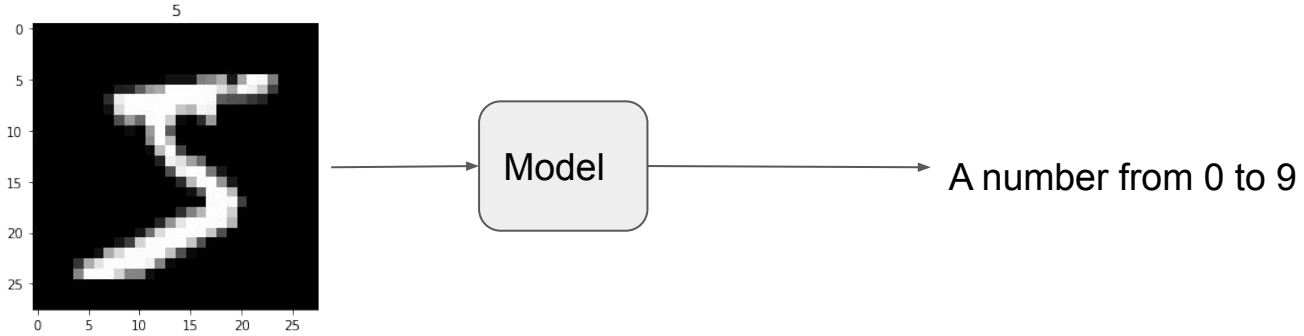
- Zero-frequency problem:
  - Zero frequency occurs when a categorical variable doesn’t exist within the training set
- Independence assumption:
  - assumes all features are independent, but this is uncommon in real life

Source: https://machinelearningmastery.com/types-of-classification-in-machine-learning/

Multiclass Classification

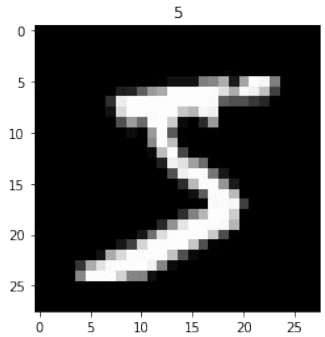
- K-nearest neighbors
- Decision trees
- Naive bayes
- Random Forest
- Gradient Boosting





Treat It as a Binary Classification Task

- One-vs-rest
- One–vs-one



Model

Is it a 0?

Model

Is it a 1?

.
.
.

Is it a 5?

Model

Is it a 8?

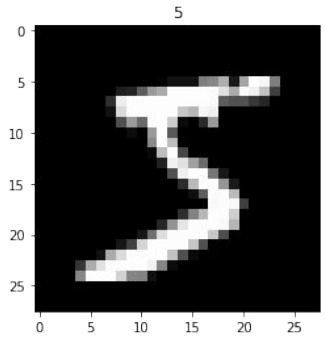
Model

Is it a 9?

Highest probability

Treat It as a Binary Classification Task

- One-vs-rest
- One–vs-one



Model — Is it 0 or 1?

Model — Is it 0 or 2?

. . .

Is it 5 or 6?

Model — Is it 7 or 9?

Model — Is it 8 or 9?

45 binary classifiers

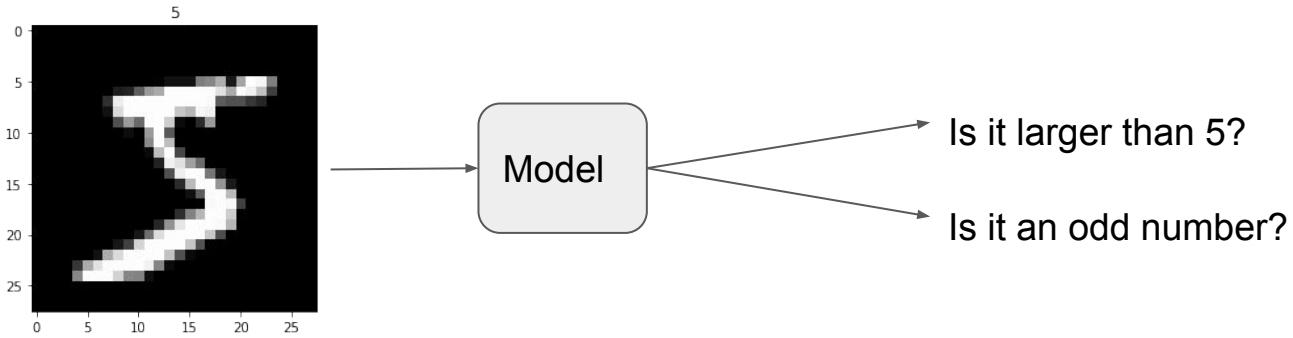
Majority count

Ex:
0: 3
1: 0
2: 7
5: 9
…

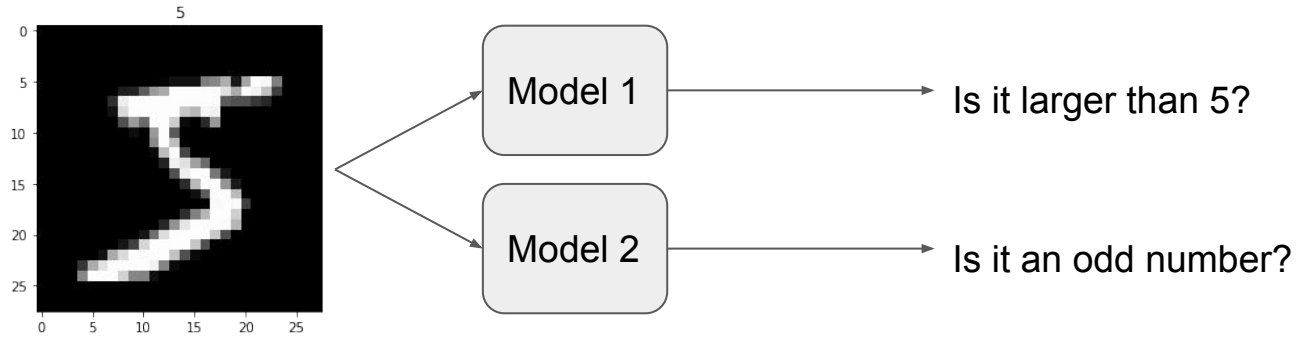
Result

Multilabel Classification

- K-nearest neighbors
- Decision tree
- Random forest

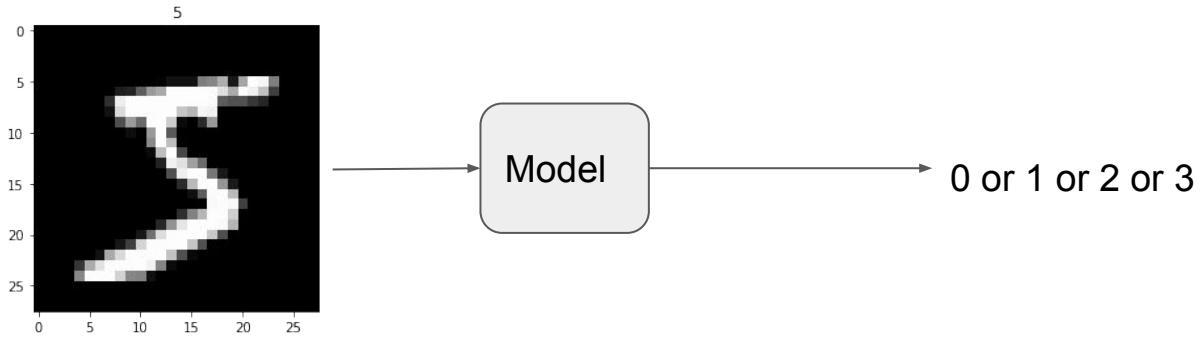


Separate Classification Algorithm for Each Label



Convert into One Label (Encoding)

Larger than 5 and odd -> 0
Larger than 5 and even -> 1
No larger than 5 and odd -> 2
No larger than 5 and even -> 3



Multioutput Classification

- K-nearest neighbors
- Decision tree
- Random forest

A 5x5 image contains 25 pixels
-> number of label

The value of each pixel ranges from 0 to 255
-> number of class



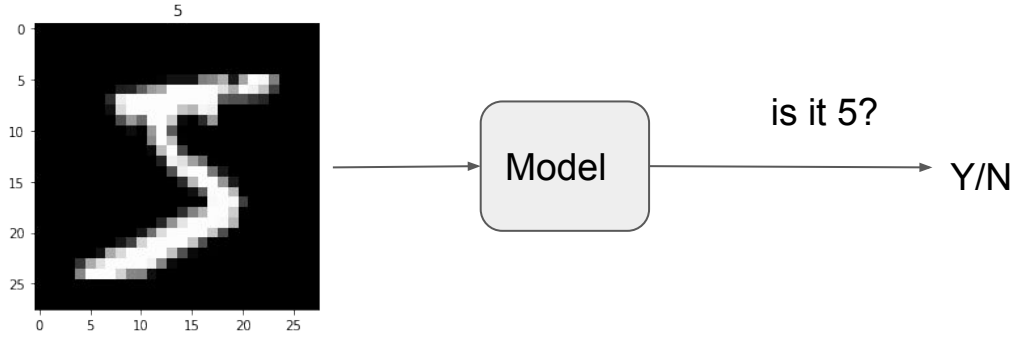
Denoising Model



[[0, 0, 0, 0,0],
[0,255,255,100,0],
[0,125,200,135,0],
[.....],

[.....]]

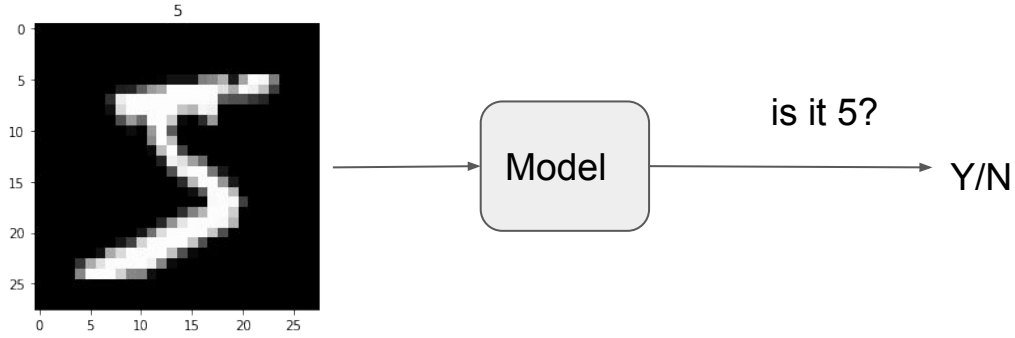
High Accuray = Good Performance?



Sample size of:

- 5 ≅ 10%
- The rest ≅ 90%

High Accuray = Good Performance?

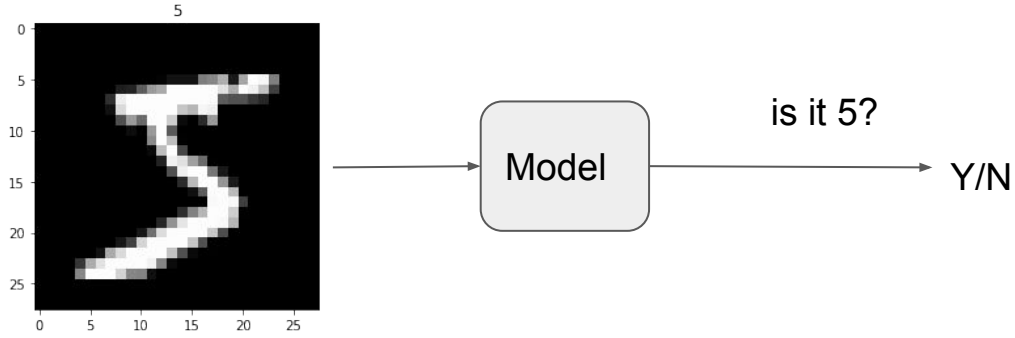


Sample size of:

- 5 ≅ 10%
- The rest ≅ 90%

→ If a dumb classifier predict every sample as “Not 5”

High Accuray = Good Performance?

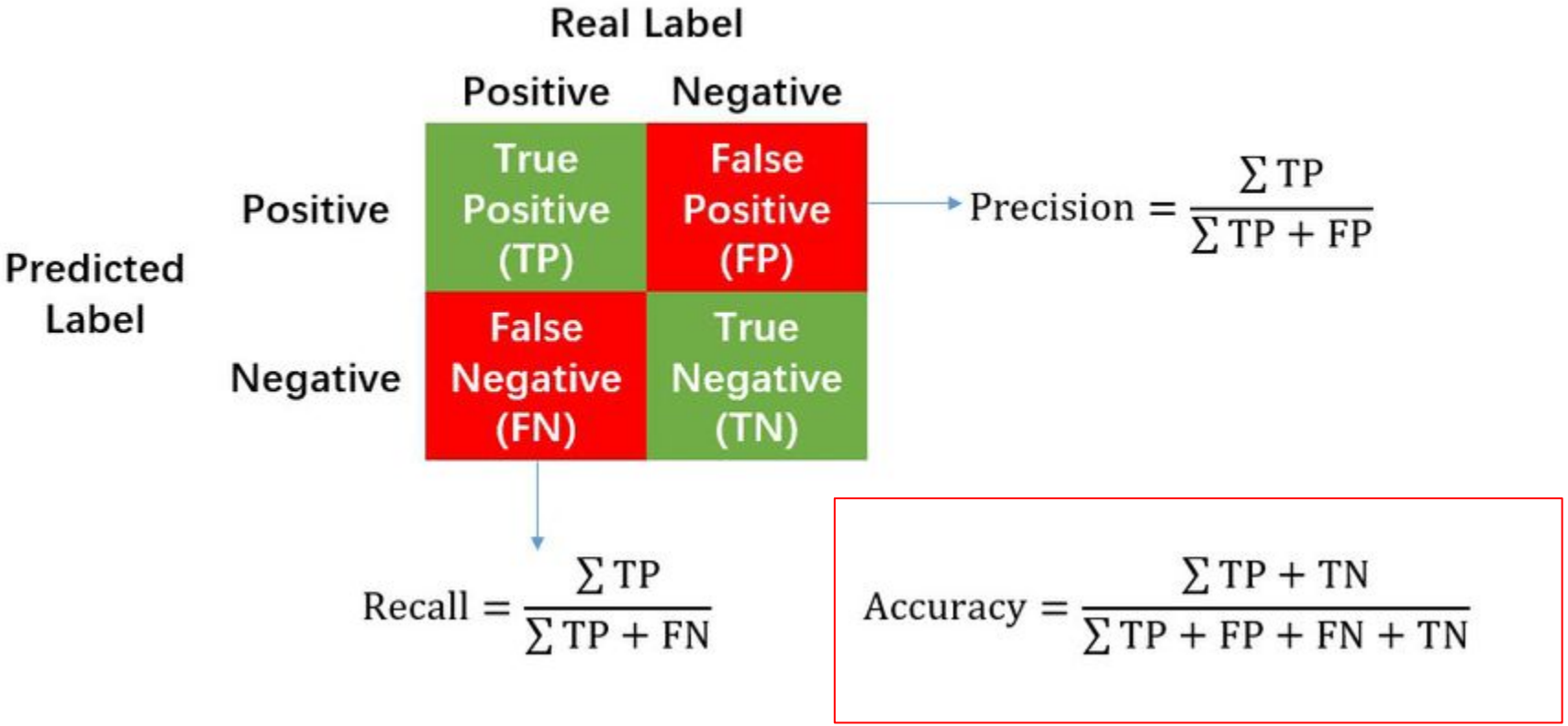


Sample size of:

- 5 ≅ 10%
- The rest ≅ 90%

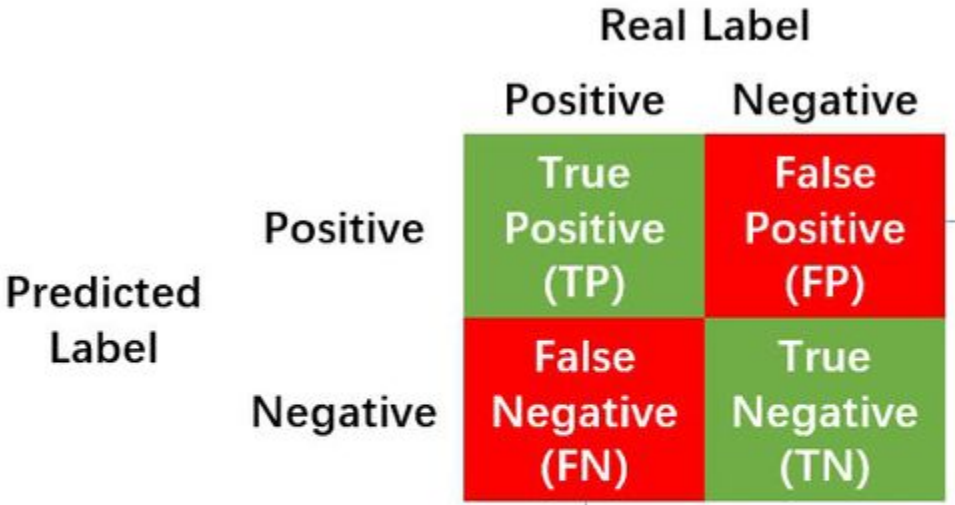
→ If a dumb classifier predict every sample as “Not 5” → Accuracy ≅ 90%

Confusion Matrix



Confusion Matrix

Of the instances we predicted to be positive, how many actually turned out to be positive?



$$\text{Precision} = \frac{\sum \text{TP}}{\sum \text{TP} + \text{FP}}$$

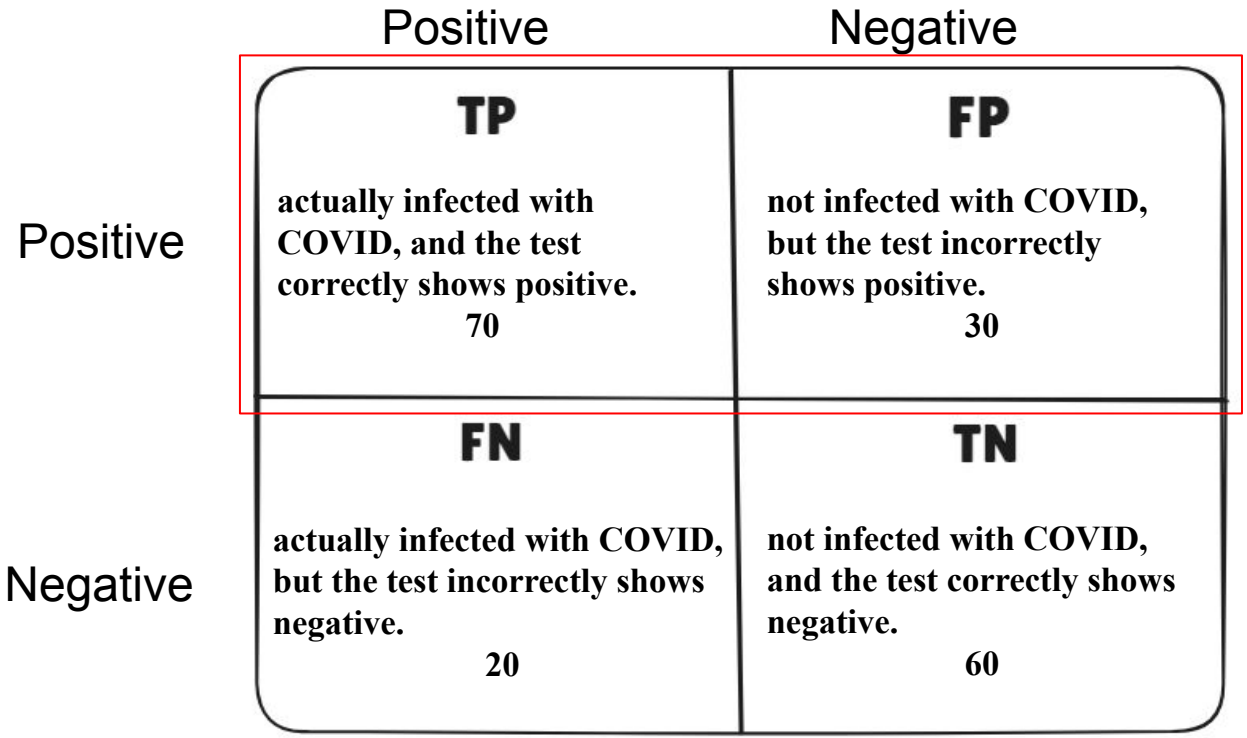
Of the instances that are actually positive, how many do we predict to be positive?

$$\text{Recall} = \frac{\sum \text{TP}}{\sum \text{TP} + \text{FN}}$$

$$\text{Accuracy} = \frac{\sum \text{TP} + \text{TN}}{\sum \text{TP} + \text{FP} + \text{FN} + \text{TN}}$$

Example

- Precision focuses on how accurate the positive predictions are

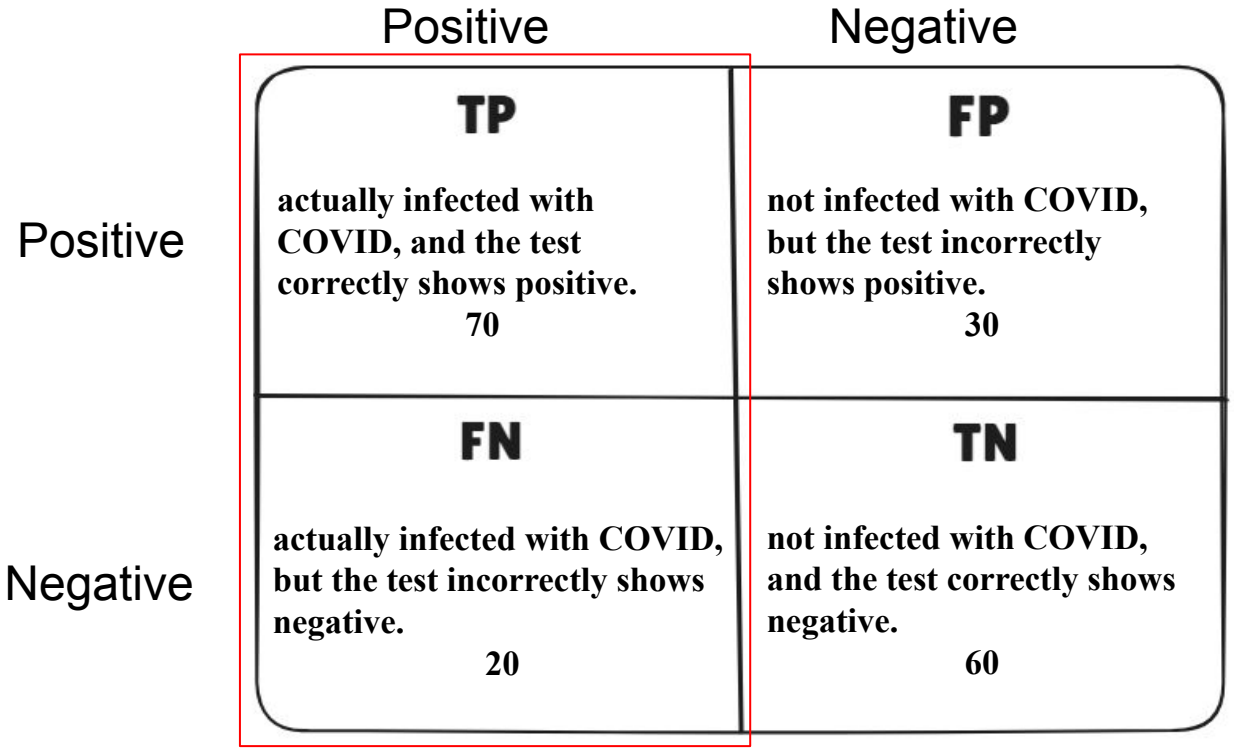


$$\text{Precision} = \frac{\sum \text{TP}}{\sum \text{TP} + \text{FP}}$$

$$\text{Precision} = \frac{70}{70 + 30} = 0.7$$

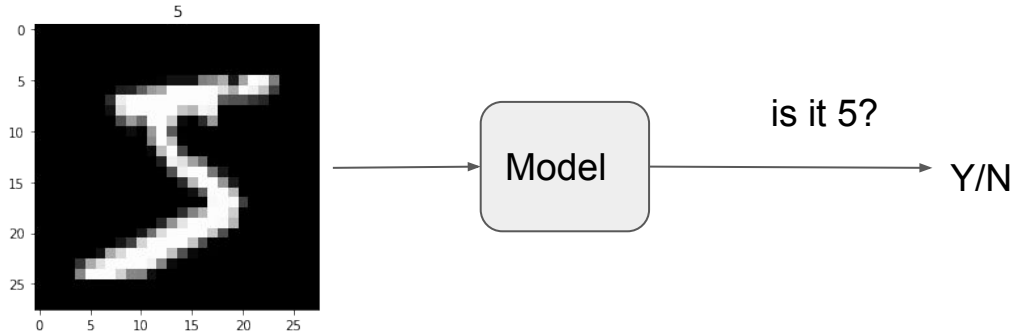
Example

- Recall focuses on how many actual cases of COVID-19 the test correctly identifies



$$\text{Recall} = \frac{\sum \text{TP}}{\sum \text{TP} + \text{FN}}$$

$$\text{Recall} = \frac{70}{70 + 20} = 0.78$$



Sample size of:

- 5 ≅ 10%
- The rest ≅ 90%

→ If a dumb classifier predict every sample as “Not 5”

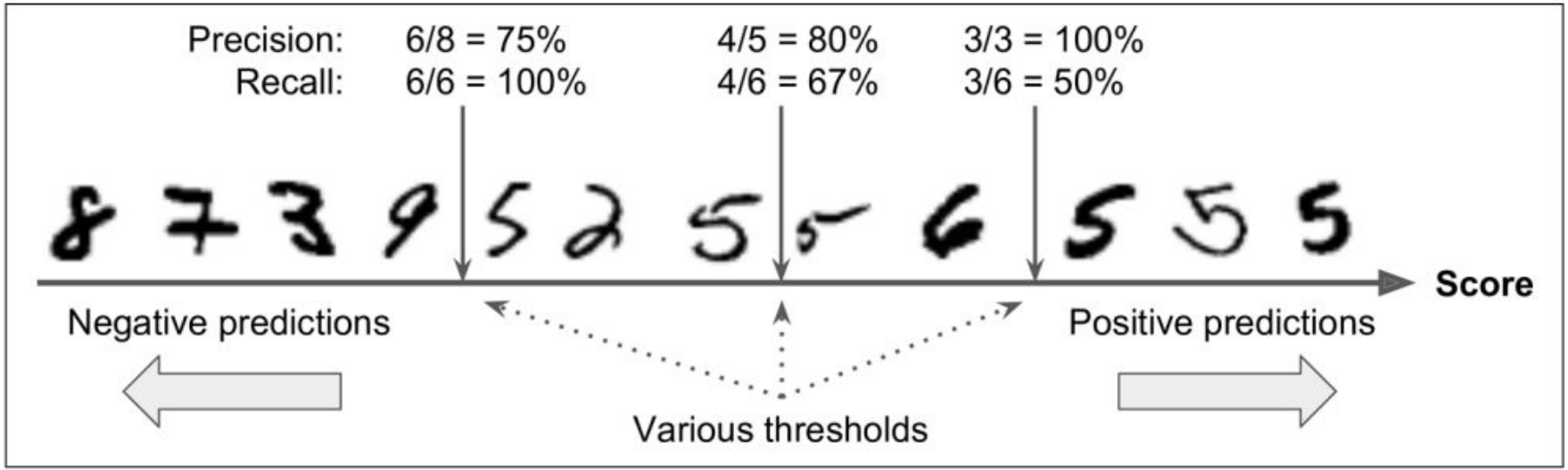
→ Accuracy ≅ 90%
Precision ≅ 0%
Recall ≅ 0%

F1

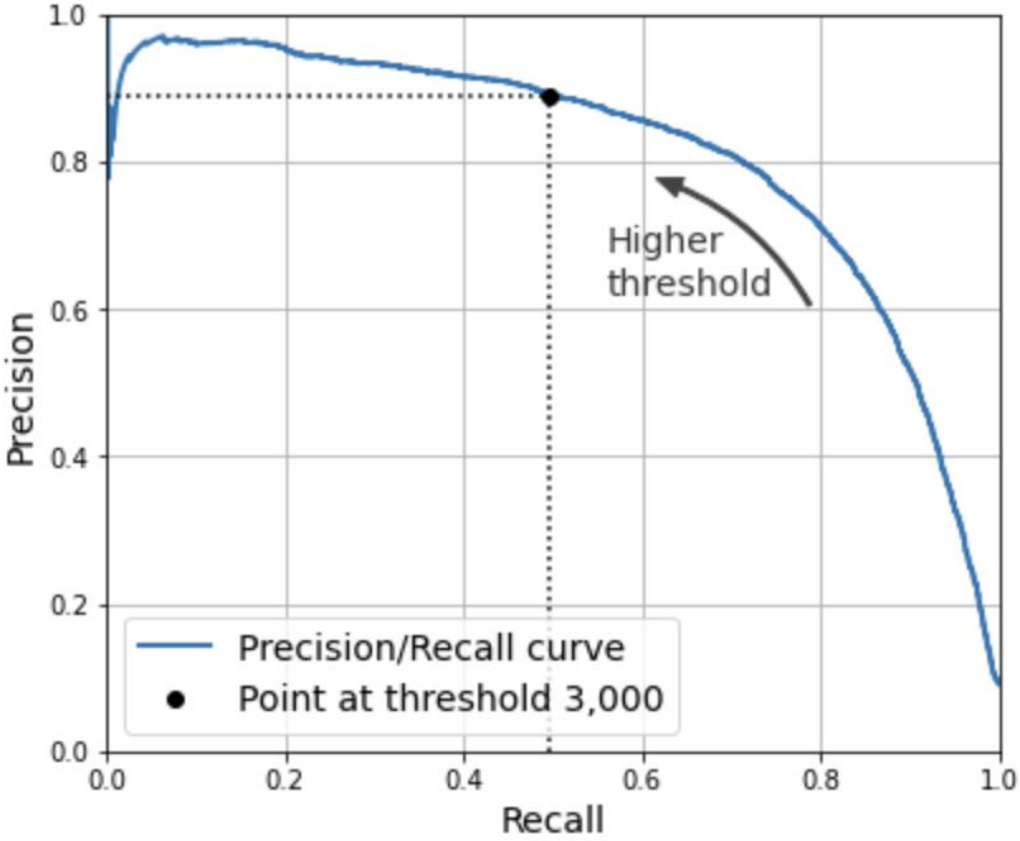
- Represent harmonic mean of precision and recall
  - The harmonic mean gives more weight to low values
- A classifier will achieve a high F1 score only if both precision and recall are high
- The F1 score favors classifiers that have balanced precision and recall, highlighting the need to find an optimal trade-off

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

Tradoff Between Precision and Recall

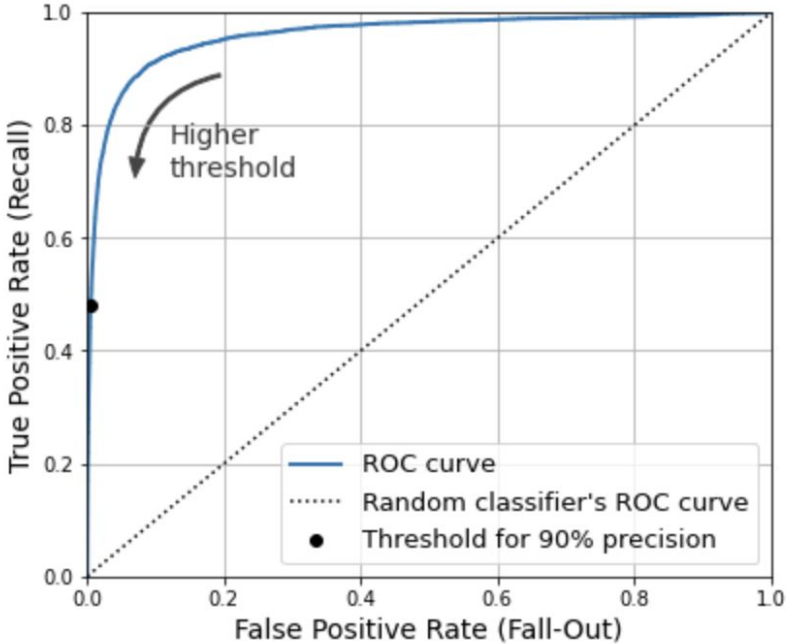


Source: Hands on Machine Learning by Geron Aurelien, page 94


1.0
0.8
0.6
0.4
0.2
0.0
Precision
0.0
0.2
0.4
0.6
0.8
1.0
Recall
Higher
threshold
Precision/Recall curve
Point at threshold 3,000

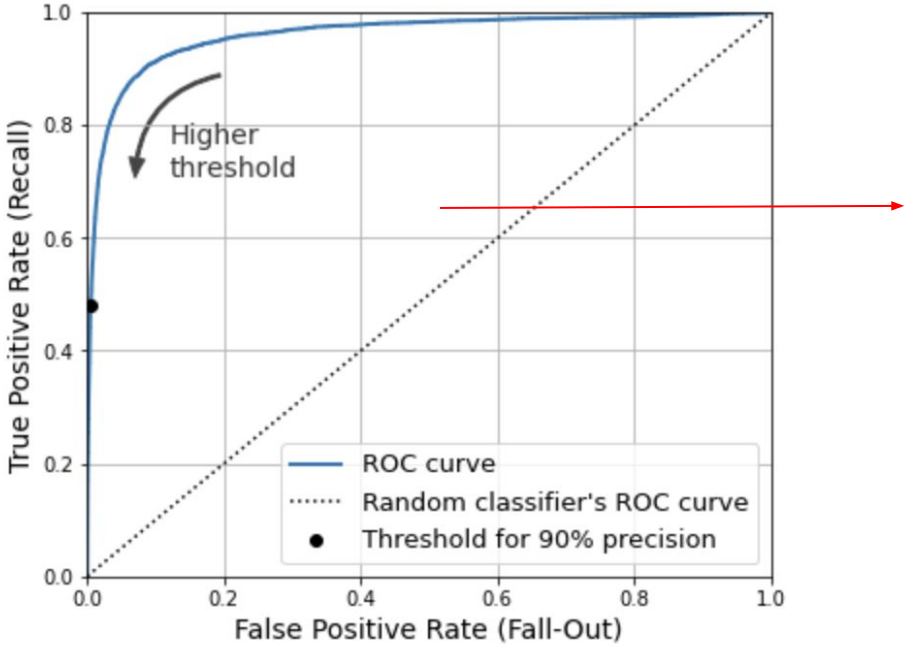
ROC Curve

- Receiver operating characteristic (ROC)
- True positive rate (recall) vs false positive rate



ROC Curve

- Receiver operating characteristic (ROC)
- True positive rate (recall) vs false positive rate



- Area under the curve, **AUC**
- Perfect classifier’s ROC AUC is 1
- Random classifier’s ROC AUC is 0.5